Technical Design Document

Benjamin Qian

May 25, 2026

Contents

1 Overview

This application is a web-based link management tool built using Angular. it allows users to add, view, and select links from a list. The goal of the system is to provide a simple interface for interacting with saved URLs.

2 Requirements

The application is a web-based bookmark management system that allows users to create, edit, and delete links. The system is fully client-side and does not use any backend database.

The application consists of two main pages:

1. Overview Page

- Provides a form for users to submit a new link.
- The form must validate that the input is a valid and reachable URL.
- Displays a paginated list of all stored links.
- Pagination must display 20 links per page.
- Pagination must include numbered navigation along with next and previous controls (e.g. <1 2 3 >).
- Allows users to edit and delete existing links.

2. Results Page

- Displays a confirmation message thanking the user for their submission.
- Shows the details of the submitted link.
- Provides navigation back to the Overview page.

Additionally the application must be built using Angular, HTML, and CSS/SCSS.

3 Design

The application follows a component based architecture as shown in Figure 1

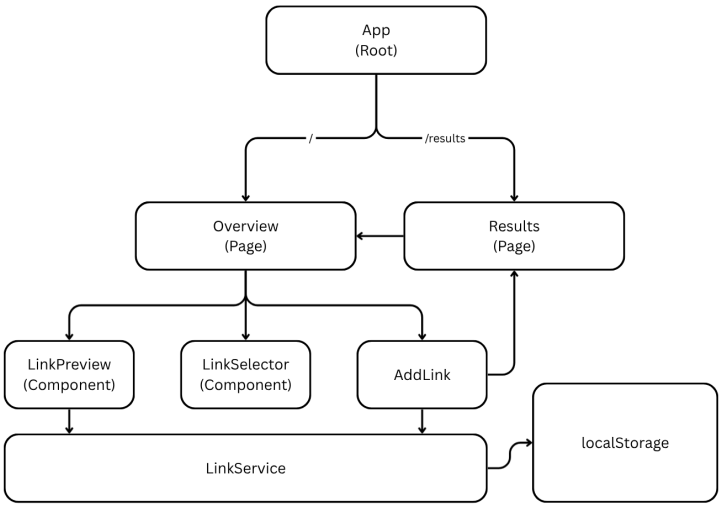

Figure 1: System Design of LinkDex

It is separated into two primary pages, the Overview page and the Results, with Angular routing used to navigate between them. The Overview page contains most of the application functionality, including the link input section, the link preview component, and the link selection component.

A dedicated LinkService is used to manage application data and handle persistence using the browser's local storage, providing a layer for storing and retrieving links.

3.1 LinkPreview and LinkSelector

The LinkPreview and LinkSelector components are implemented as separate components to improve code organisation and maintainability. Although they are not designed for reuse across multiple parts of the application, separating them into distinct components helps keep the Overview page modular and easier to manage.

This separation ensures that each component is responsible for a single concern: the LinkPreview component handles the display of individual link data, while the LinkSelector component manages user selection and interaction logic.

3.2 Data Management

The application uses local storage for data persistence due to its simplicity and suitability for a frontend-only application. Local storage retains data even after the browser is closed, making it ideal for storing user-created links without requiring a backend.

Session storage was another option. This was quickly ruled out, as it only stores data for the duration of the current session, meaning all saved links would be lost once the user closes the browser tab. This makes it unsuitable for the intended purpose, although it would fit the requirements.

3.3 Result Page

The navigation from Overview page to the Results page is implemented as a Router State to pass data. Instead of embedding the data in the URL, the application uses navigation state.

This decision was chosen because it prevents unnecessary data from appearing in the URL, improving readability, and avoids clutter. Although Router State does not persist data across page refreshes, this limitation is acceptable as the Results page is intended for immediate display of the submitted link rather than long term access.

4 Limitations

4.1 Design

A key limitation of the application is the interaction model used for viewing link details. Each favicon must be individually selected in order to view its full information. This design choice can become inefficient when a large number of links are displayed, as it requires repeated navigation between the overview and results views.

This issue is further amplified when multiple links share the same favicon, as it increases visual clutter and makes it more difficult for users to quickly identify relevant items. The lack of sorting functionality also contributes to reduced usability, as users are unable to organise links based on a criteria.

As a result, the current interface is less optimal for larger datasets, where more advanced interaction features such as sorting, filtering, or inline previews would significantly improve usability and efficiency.

4.2 Link Detection

Link detection is implemented using a set of regular expressions that perform basic pattern matching to identify whether a given string resembles a URL (e.g. begins with `http://` or `https://`, or contains URL-like characters).

However, this approach only validates the format of the input and does not verify whether the link actually resolves to a reachable or valid destination. In other words, it cannot determine if the URL leads to an existing webpage or returns a successful response from a server.

4.3 Storage

The application uses the browser's `localStorage` API to persist link data on the client side. This allows the application to function without a backend database, as all stored links remain available between page reloads in the same browser.

Each link entry is serialized as a JSON string before being stored, and deserialized back into JavaScript objects when retrieved. The full list of links is typically stored under a single key in `localStorage`, and is updated whenever a link is added, edited, or deleted.

Data is stored per browser and per device, meaning it is not shared across users or synchronized between sessions. In addition, storage capacity is limited and data can be cleared by the user at any time.